Mapping and Data Visualization

with Python

Press Space for next page →

Libraries

Folium

PHILADELPHIA

What is Leafmap?

Leafmap is a Python package for interactive mapping and geospatial analysis with minimal coding in Jupyter environments (Jupyter Notebook, JupyterLab, Google Colab).

- **Easy** - simple API for creating interactive maps
- **Powerful** - built on top of ipyleaflet and folium
- **Flexible** - works in Jupyter notebooks and web apps

Pre-requisites

1. Install Python:
   - Download from python.org
2. Install required libraries

```
pip install leafmap geopandas pandas jupyterlab
```

3. Download the workshop files
4. Launch and test Jupyter Lab

- navigate to the downloaded folder

```
cd Desktop/workshop-files
```

- Launch jupyter lab

```
jupyter lab
```

Getting Started

```
import leafmap

m = leafmap.Map(center=[40, -100], zoom=4)
m
```

Line 3: Return the "m" element (map)

```
m
```

Prints the results

leafmap’s Unique Strengths

What leafmap does that nothing else does as easily

What leafmap Does Best

Strength	Why leafmap?
Cloud-Optimized GeoTIFFs (COG)	Stream satellite imagery directly from S3/cloud — no download
Split-panel comparison	Before/after view with a draggable divider — one method call
WMS tile services	Load live layers from government and research servers
STAC catalog access	Browse and visualize spatiotemporal asset catalogs
Linked multi-panel maps	Synchronized views for comparing multiple datasets

> Open `leafmap_demo.ipynb` to follow along

1. Cloud-Optimized GeoTIFFs (COG)

A **COG** is a satellite image stored on a cloud server that lets you view any part of it without downloading the whole file.

```
url_after = "https://github.com/opengeos/datasets/releases/download/raster/Libya-2023-09-13.tif"

m = leafmap.Map(center=[32.75, 22.63], zoom=13)
m.add_cog_layer(url_after, name="Libya — Post-flood (Sep 2023)")
m
```

> `m` renders the interactive map inline in the notebook

[3]:

```
# Load a single COG — satellite imagery of Libya after the 2023 flooding
# COG files stream from the cloud — no full download needed
url_after = "https://github.com/opengeos/datasets/releases/download/raster/Libya-2023-09-13.tif"

m = leafmap.Map(center=[32.75, 22.63], zoom=13)
m.add_cog_layer(url_after, name="Libya — Post-flood (Sep 2023)")
m
```

2. Split-Panel Comparison Map

`split_map()` creates a side-by-side view with a **draggable divider** — drag left/right to compare.

```
url_before = "https://github.com/opengeos/datasets/releases/download/raster/Libya-2023-07-01.tif"
url_after  = "https://github.com/opengeos/datasets/releases/download/raster/Libya-2023-09-13.tif"

m = leafmap.Map(center=[32.75, 22.63], zoom=13, height=500)
m.split_map(
    left_layer=url_before,
    right_layer=url_after,
    left_label="Before — July 2023",
    right_label="After — September 2023 (flooding)"
)
m
```

`split_map()` also works with **basemap names** — no COG URL needed:

```
m.split_map(left_layer="Esri.WorldImagery", right_layer="CartoDB.Positron")
```

```python
url_before = "https://github.com/opengeos/datasets/releases/download/raster/Libya-2023-07-01.tif"
url_after  = "https://github.com/opengeos/datasets/releases/download/raster/Libya-2023-09-13.tif"

m = leafmap.Map(
    center=[32.75, 22.63],
    zoom=13,
    height=500
)
m.split_map(
    left_layer=url_before,
    right_layer=url_after,
    left_label="Before — July 2023",
    right_label="After — September 2023 (flooding)"
)
m
```

3. WMS Tile Services

WMS (Web Map Service) is a standard protocol used by governments and research institutions to serve live map data. leafmap loads any WMS layer in two lines.

```
# NLCD 2021 Land Cover — live from the USGS/MRLC server
wms_url = "https://www.mrlc.gov/geoserver/mrlc_display/NLCD_2021_Land_Cover_L48/wms?"

m = leafmap.Map(center=[40, -100], zoom=4)
m.add_wms_layer(
    wms_url,
    layers="NLCD_2021_Land_Cover_L48",
    name="NLCD 2021 Land Cover",
    format="image/png",
    transparent=True
)
m.add_legend(builtin_legend="NLCD", title="Land Cover Type")
m
```

> `add_legend(builtin_legend="NLCD")` — leafmap ships with NLCD and other built-in legends

```python
wms_url = "https://www.mrlc.gov/geoserver/mrlc_display/NLCD_2021_Land_Cover_L48/wms?"

m = leafmap.Map(center=[40, -100], zoom=4)
m.add_wms_layer(
    wms_url,
    layers="NLCD_2021_Land_Cover_L48",
    name="NLCD 2021 Land Cover",
    format="image/png",
    transparent=True
)
# Add the built-in NLCD legend
m.add_legend(builtin_legend="NLCD", title="Land Cover Type")
m
```

4. Land Cover Change — WMS Split Map

leafmap has NLCD pre-configured as built-in basemap names. Combine with `split_map()` to show **20 years of land cover change** — no WMS setup needed.

```
m = leafmap.Map(center=[36.1, -114.9], zoom=9, height=500)
m.split_map(
    left_layer="NLCD 2001 CONUS Land Cover",
    right_layer="NLCD 2021 CONUS Land Cover",
    left_label="2001",
    right_label="2021"
)
m.add_legend(builtin_legend="NLCD", title="Land Cover")
m
```

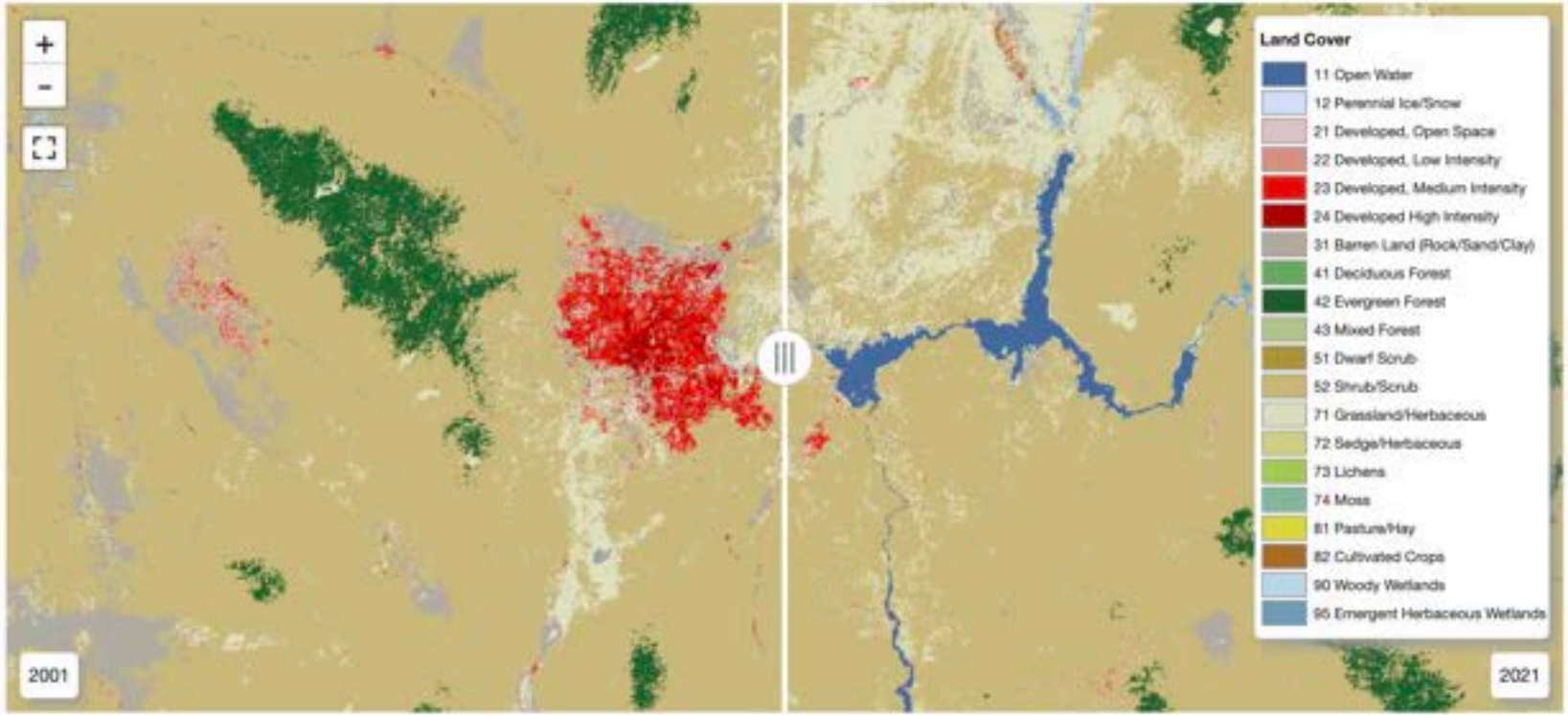
Left panel — 2001

The landscape before rapid development around Las Vegas, NV

Right panel — 2021

Urban sprawl visible as gray/developed land cover expanded

[6]:
```
# Compare NLCD 2001 vs 2021 land cover using built-in leafmap basemap names
# These NLCD layers are pre-configured in leafmap - no WMS setup needed

m = leafmap.Map(center=[36.1, -114.9], zoom=9, height=500)
m.split_map(
    left_layer="NLCD 2001 CONUS Land Cover",
    right_layer="NLCD 2021 CONUS Land Cover",
    left_label="2001",
    right_label="2021"
)
m.add_legend(builtin_legend="NLCD", title="Land Cover")
m
```



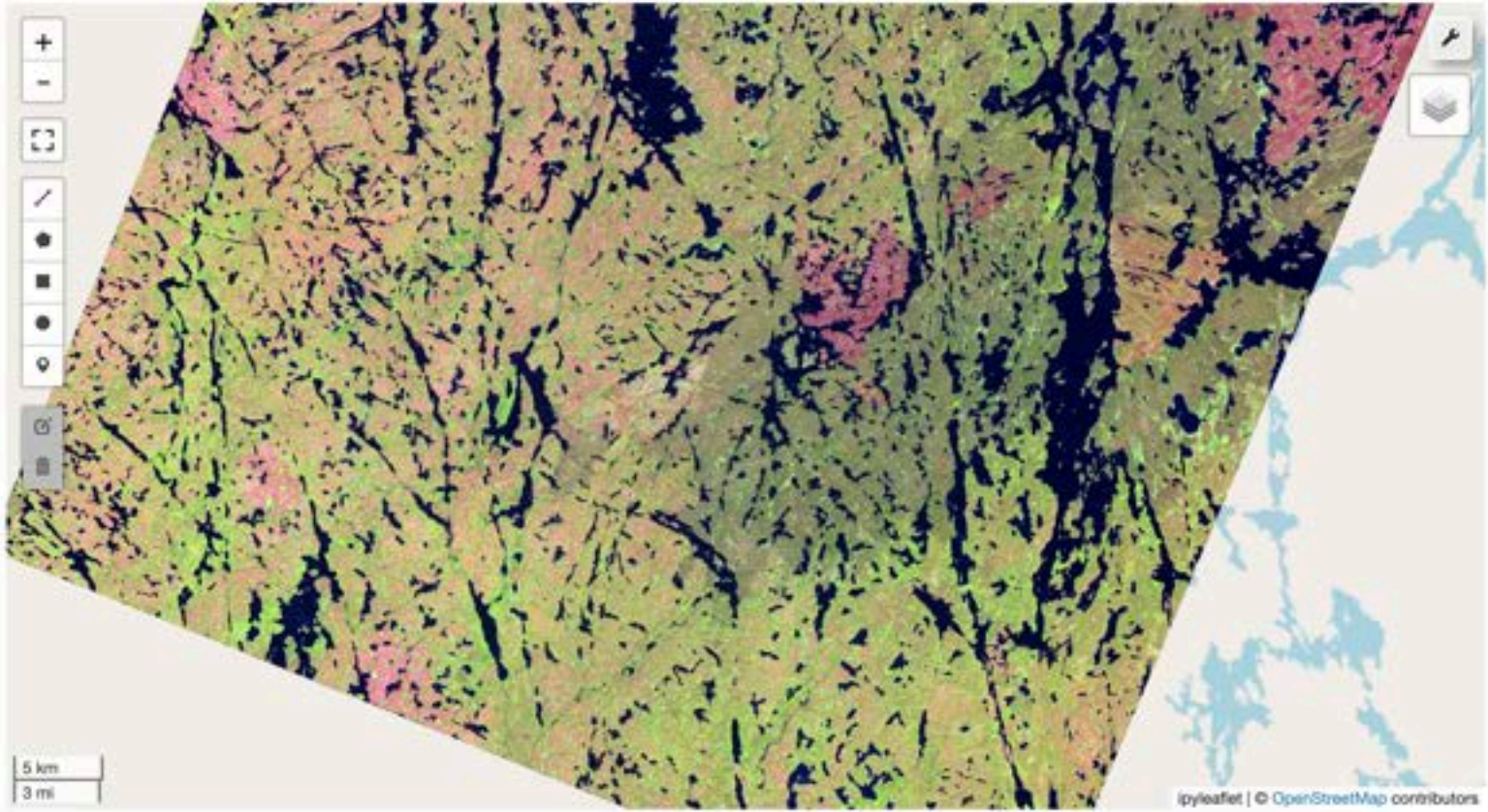
5. STAC — SpatioTemporal Asset Catalog

STAC is a standard for organizing petabytes of satellite imagery. NASA, Microsoft Planetary Computer, and AWS all host STAC catalogs. leafmap loads a STAC item directly from a URL.

```
stac_url = (
    "https://canada-spot-ortho.s3.amazonaws.com/canada_spot_orthoimages/"
    "canada_spot5_orthoimages/S5_2007/S5_11055_6057_20070622/S5_11055_6057_20070622.json"
)

print("Available bands:", leafmap.stac_bands(stac_url))
# → ['pan', 'B1', 'B2', 'B3', 'B4']

m = leafmap.Map(center=[60.57, -110.55], zoom=10)
m.add_stac_layer(stac_url, bands=["B3","B2","B1"], name="True Color (RGB)", vmin=0, vmax=150)
m.add_stac_layer(stac_url, bands=["B4","B3","B2"], name="False Color (NIR)", shown=False)

m.add_layer_control()
m
```

> `shown=False` loads the layer but keeps it hidden — toggle it on via the layer control.
> False color (NIR) highlights vegetation in bright red.

```python
m.add_stac_layer(stac_url, bands=["B3", "B2", "B1"], name="True Color (RGB)", vmin=0, vmax=150)

# False color — near-infrared highlights vegetation in red
# shown=False loads the layer but keeps it hidden until toggled on
m.add_stac_layer(stac_url, bands=["B4", "B3", "B2"], name="False Color (NIR)", vmin=0, vmax=150, shown=False)

# Layer control lets you toggle between True Color and False Color
m.add_layer_control()
m
```



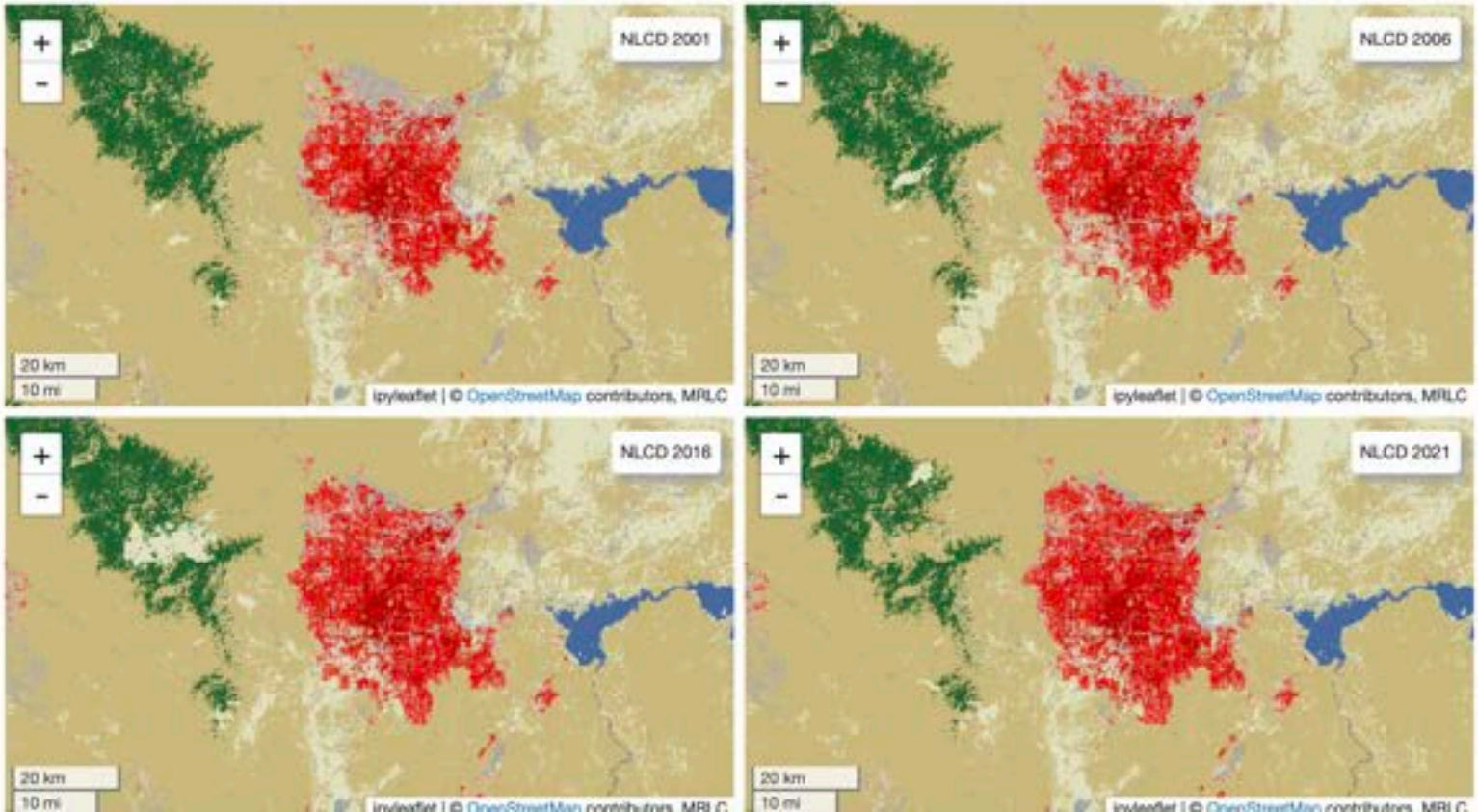
6. Linked Multi-Panel Maps

`leafmap.linked_maps()` creates multiple maps that are **synchronized** — pan or zoom one and all others follow.

```
layers = [f"NLCD {year} CONUS Land Cover" for year in [2001, 2006, 2016, 2021]]
labels = [f"NLCD {year}" for year in [2001, 2006, 2016, 2021]]

leafmap.linked_maps(
    rows=2,
    cols=2,
    height="300px",
    layers=layers,
    labels=labels,
    center=[36.1, -115.2],
    zoom=9
)
```

Four NLCD snapshots in a 2×2 grid — zoom into any panel and the others jump to the same location.
Great for teaching land cover change or comparing multiple scenarios.

```
leafmap.linked_maps(
    rows=2,
    cols=2,
    height="300px",
    layers=layers,
    labels=labels,
    center=[36.1, -115.2],
    zoom=9
)
```



leafmap vs Folium — When to Use Which

Use case	leafmap	Folium
Satellite imagery (COG/STAC)	✅ Best choice	❌ Not supported
Before/after split map	✅ One line	⚠️ Complex setup
WMS tile services	✅ Simple	✅ Works
Linked synchronized maps	✅ Built-in	❌ Not supported
Vector data (points, polygons)	⚠️ Works in notebook	✅ Best choice
Choropleth maps	⚠️ Works but quirky	✅ Best choice
HTML export for web	⚠️ Widget-based, fragile	✅ Clean Leaflet.js

Part 2: Folium Workshop

Philadelphia Public Art — Building an Interactive Web Map

Step 1 — Create an Empty Map

```
m = folium.Map()
m
```

`m` on its own line tells Jupyter to display the map widget.
Unlike leafmap, `m` here is a pure HTML object — no widgets needed.

Step 2 — Center, Zoom, and Basemap

```
m = folium.Map(
    location=[39.952, -75.165],   # Philadelphia center
    zoom_start=12,                # city-level zoom
    tiles="CartoDB Positron"      # clean light basemap
)
m
```

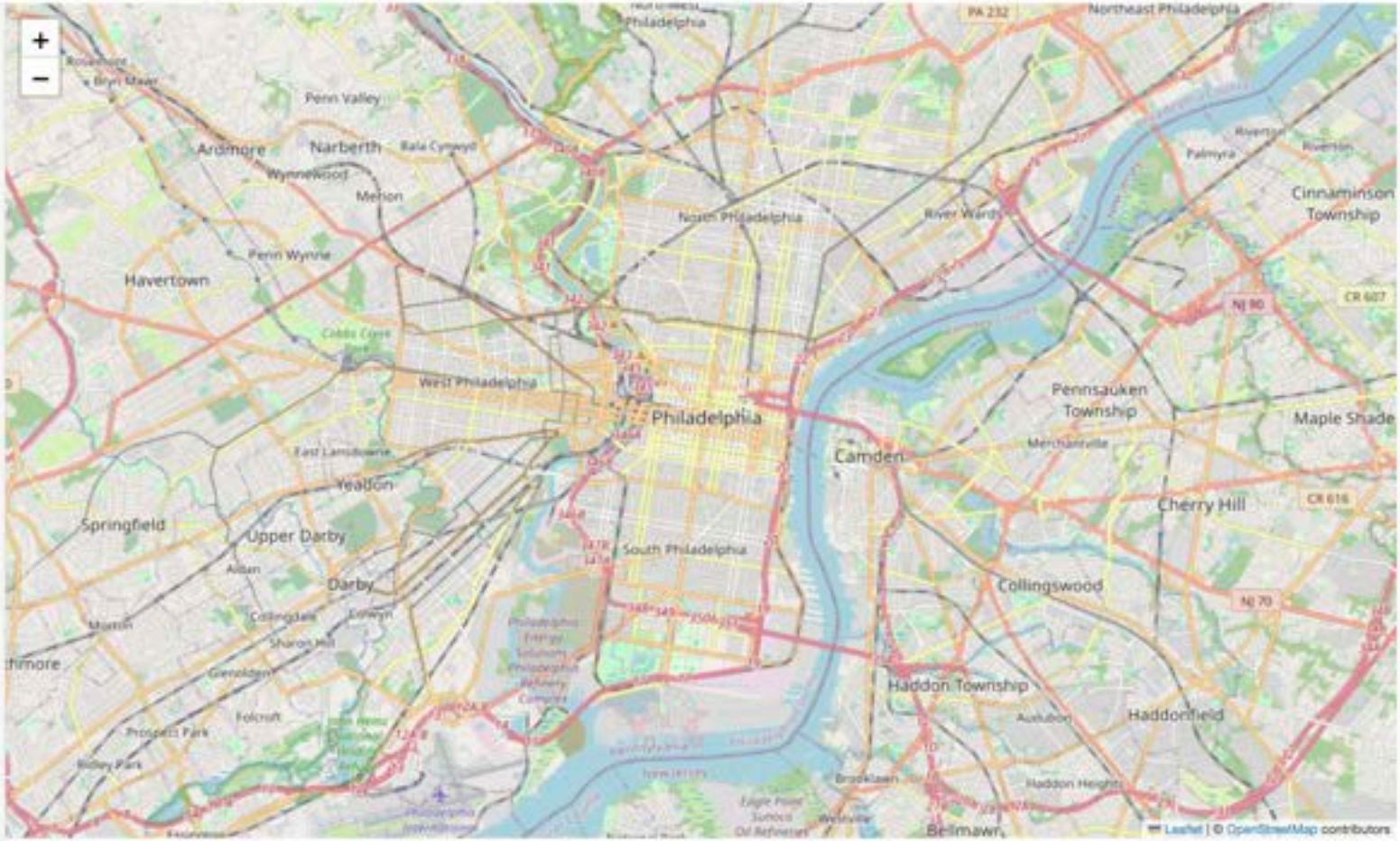
Common basemap options:

`tiles=` value	Style
`"OpenStreetMap"`	Detailed street map (default)
`"CartoDB Positron"`	Clean light — best for data visualization
`"CartoDB DarkMatter"`	Dark background

[3]:
```
m = folium.Map(
    location=[39.952, -75.165],
    zoom_start=12
)
m
```

Collapse Output

[3]:



Step 3 — Load a GeoJSON Layer

```
neighborhoods_url = (
    "https://raw.githubusercontent.com/opendataphilly/open-geo-data/master/"
    "philadelphia-neighborhoods/philadelphia-neighborhoods.geojson"
)

m = folium.Map(location=[39.952, -75.165], zoom_start=12, tiles="CartoDB Positron")

folium.GeoJson(
    neighborhoods_url,
    name="Neighborhoods",
    style_function=lambda x: {
        "color": "#555555",
        "weight": 0.8,
        "fillOpacity": 0.0
    }
).add_to(m)
```

> `style_function` is a lambda that returns a style dict for each feature.
>
> `fillOpacity: 0.0` shows only outlines — no fill color yet.

[6]:

```python
neighborhoods_url = (
    "https://raw.githubusercontent.com/opendataphilly/open-geo-data/master/"
    "philadelphia-neighborhoods/philadelphia-neighborhoods.geojson"
)

m = folium.Map(location=[39.952, -75.165], zoom_start=12, tiles="CartoDB Positron")

# Add neighborhood boundaries as a styled GeoJSON layer
folium.GeoJson(
    neighborhoods_url,
    name="Neighborhoods",
    style_function=lambda x: {
        "color": "#555555",
        "weight": 0.8,
        "fillOpacity": 0.0
    }
).add_to(m)

m
```

[6]:

Step 4 — Load Point Data from CSV

```
art_df = pd.read_csv("philly_public_art.csv")
art_df = art_df.dropna(subset=["latitude", "longitude"])
art_df["year"] = pd.to_numeric(art_df["year"], errors="coerce")

for _, row in art_df.iterrows():
    folium.CircleMarker(
        location=[row["latitude"], row["longitude"]],
        radius=4,
        color="#333333",
        fill=True,
        fill_color="#e8a320",
        fill_opacity=0.8
    ).add_to(m)
```

Dataset: 534 Philadelphia artworks

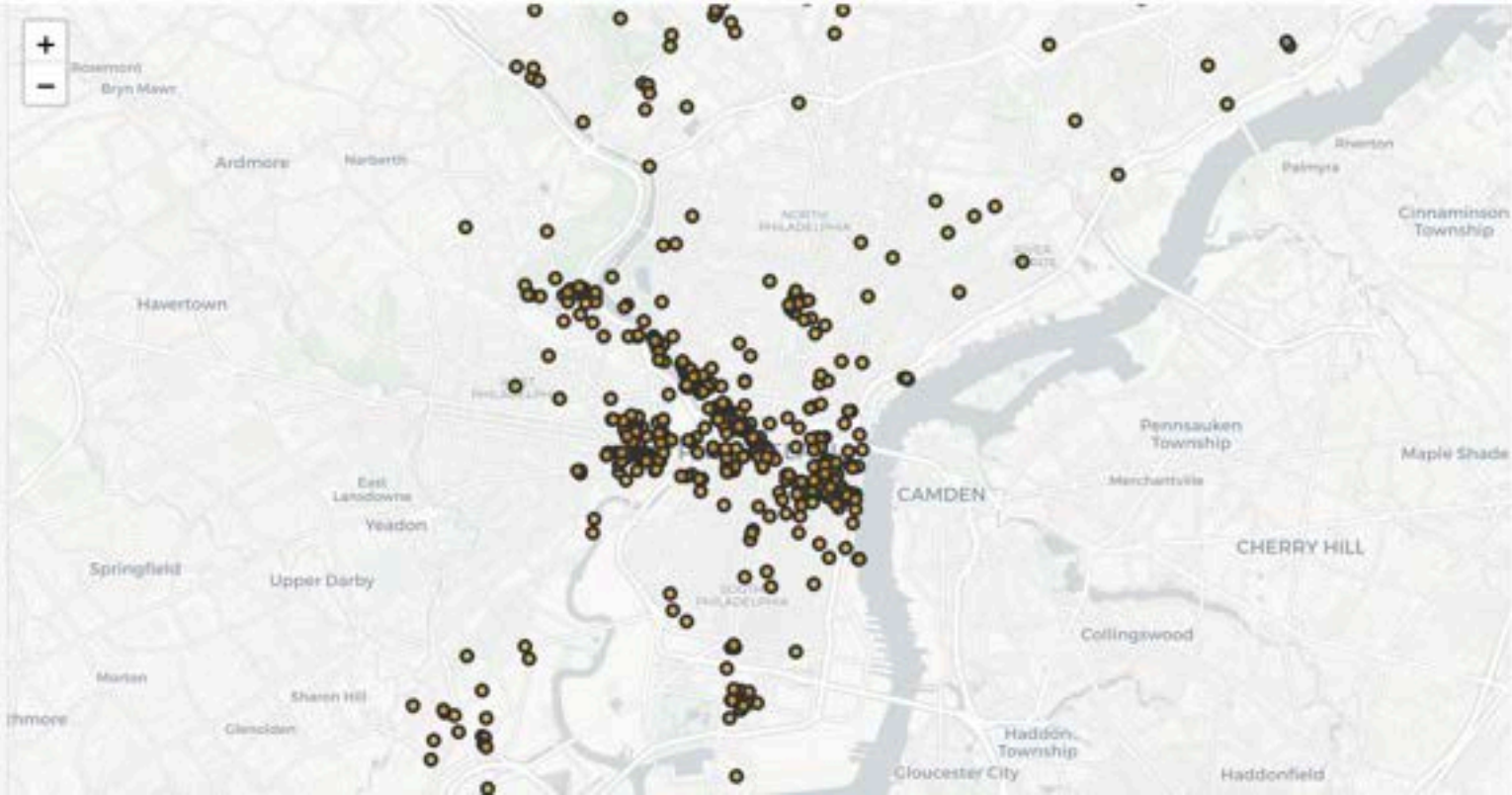
Columns: `title`, `artist`, `year`, `type`, `latitude`, `longitude`

Year range: 1400 – 2024

`CircleMarker` draws a fixed-pixel circle at a lat/lon.

Unlike `Marker` (pin icon), it scales well with many points.

```python
for _, row in art_df.iterrows():
    folium.CircleMarker(
        location=[row["latitude"], row["longitude"]],
        radius=4,
        color="#333333",
        fill=True,
        fill_color="#e8a320",
        fill_opacity=0.8
    ).add_to(m)

m
```



Step 5 — Color Points by Decade

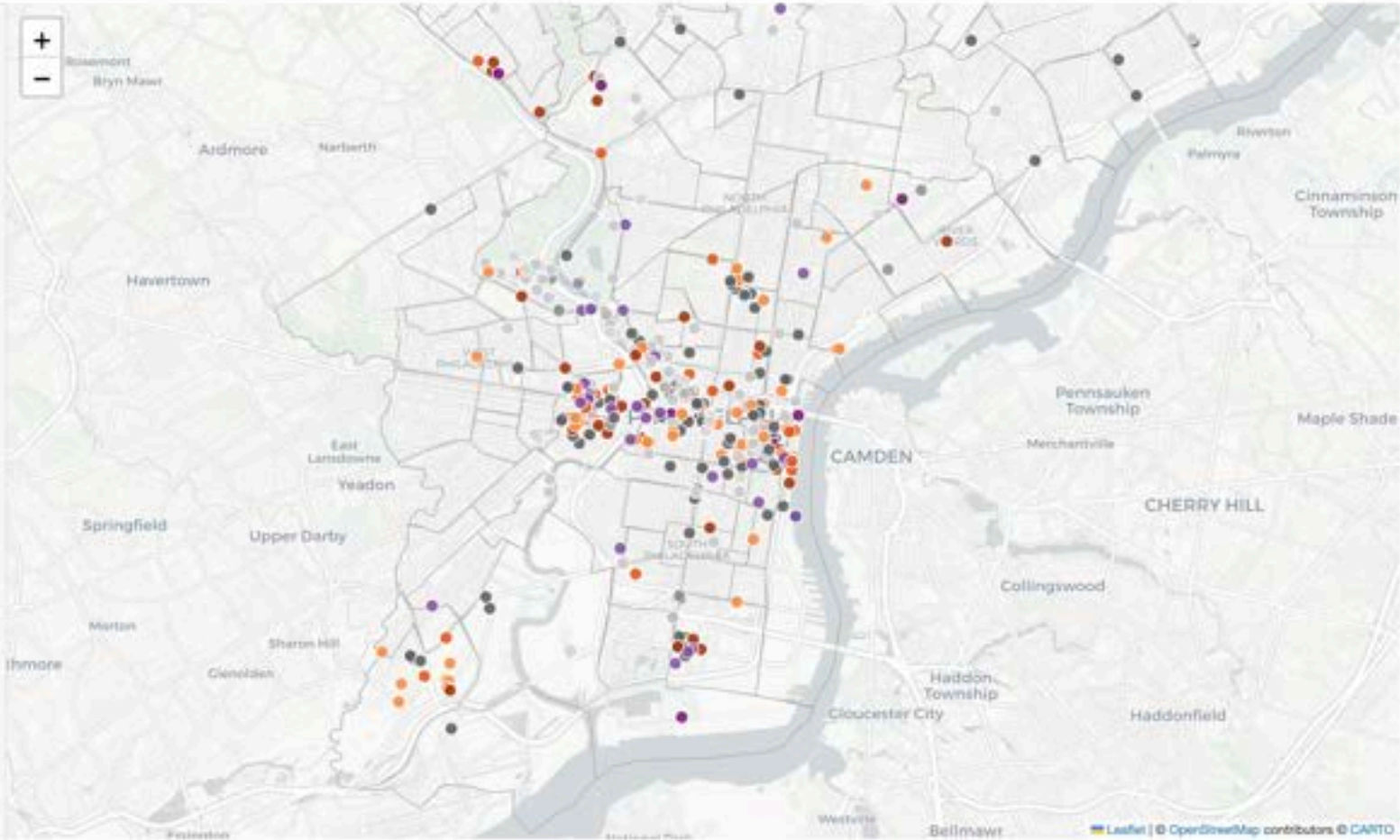
In Folium, coloring by attribute is simple: compute the color in Python, pass it to `fill_color`.

```
DECADE_COLORS = {
    1980: "#fd8d3c",   # Mural Arts program founded
    1990: "#e6550d",
    2000: "#a63603",
    2010: "#8856a7",
    2020: "#810f7c",
}

def get_decade_color(year):
    ...   # compute decade, look up in DECADE_COLORS

art_df["color"] = art_df["year"].apply(get_decade_color)

for _, row in art_df.iterrows():
    folium.CircleMarker(..., fill_color=row["color"], ...).add_to(m)
```

Why 1980? Philadelphia's Mural Arts Program launched in 1984, sparking a wave of public art. The orange/red colors highlight this era.

```
        fill=True,
        fill_color=row["color"],
        fill_opacity=0.9
    ).add_to(m)
```

[10]:



Step 6 — Add Popups and Tooltips

```
for _, row in art_df.iterrows():
    popup_html = f"""
        <div style="font-family: Arial; font-size: 12px; width: 180px;">
            <b style="color: #3d0059;">{row['title']}</b><br>
            <span style="color: #555;">{artist}</span><br>
            <span style="color: #888;">{year}</span>
        </div>
    """
    folium.CircleMarker(
        ...,
        popup=folium.Popup(popup_html, max_width=200),
        tooltip=row["title"]
    ).add_to(m)
```

`popup` — HTML shown on **click**.

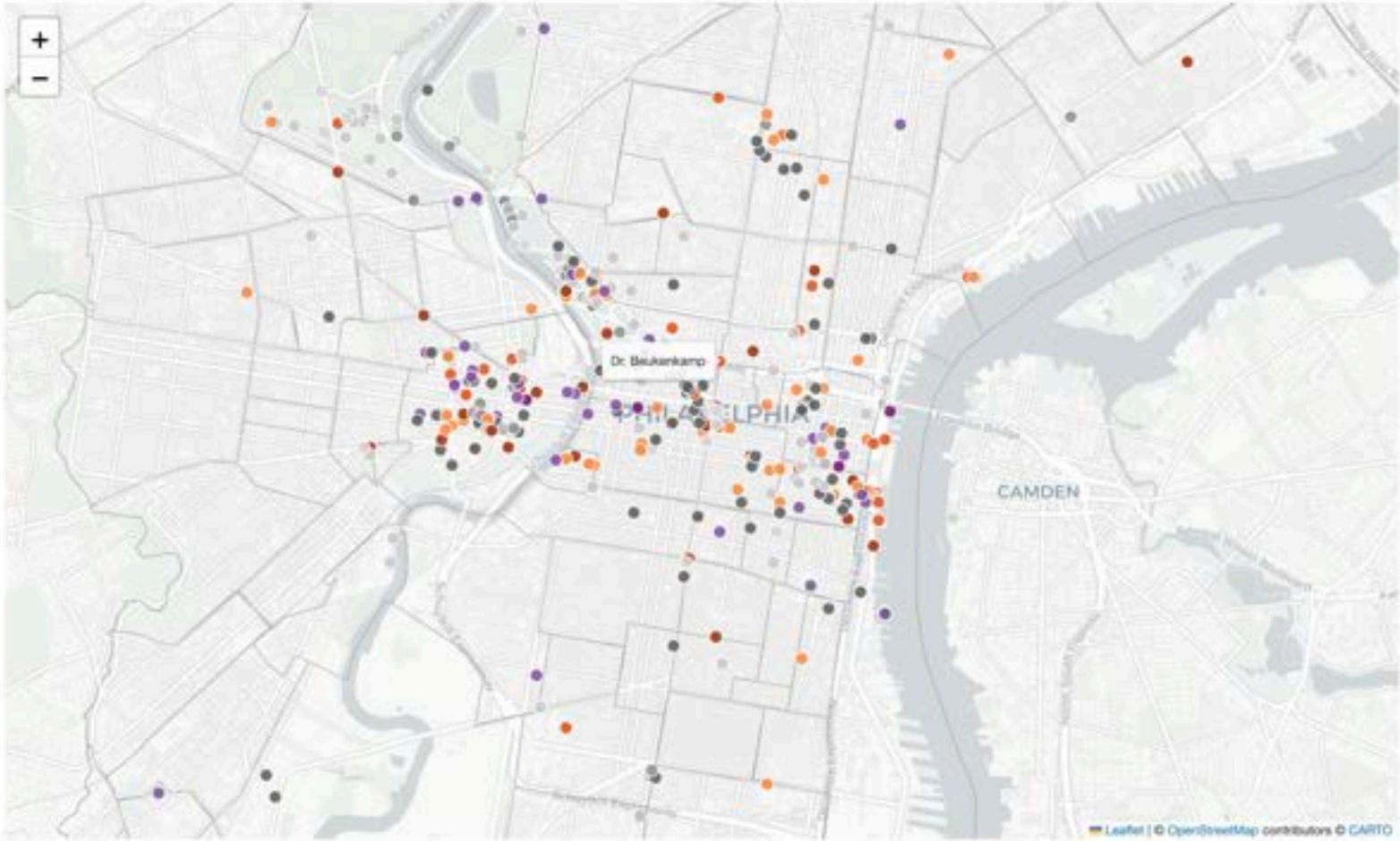
`folium.Popup(html, max_width=...)` wraps the string.

`tooltip` — plain text on **hover**.

Keep it short — just the artwork title.

```
        popup=folium.Popup(popup_html, max_width=200),
        tooltip=row["title"]   # shows on hover without clicking
    ).add_to(m)
```

[11]:



Step 7 — Choropleth Map

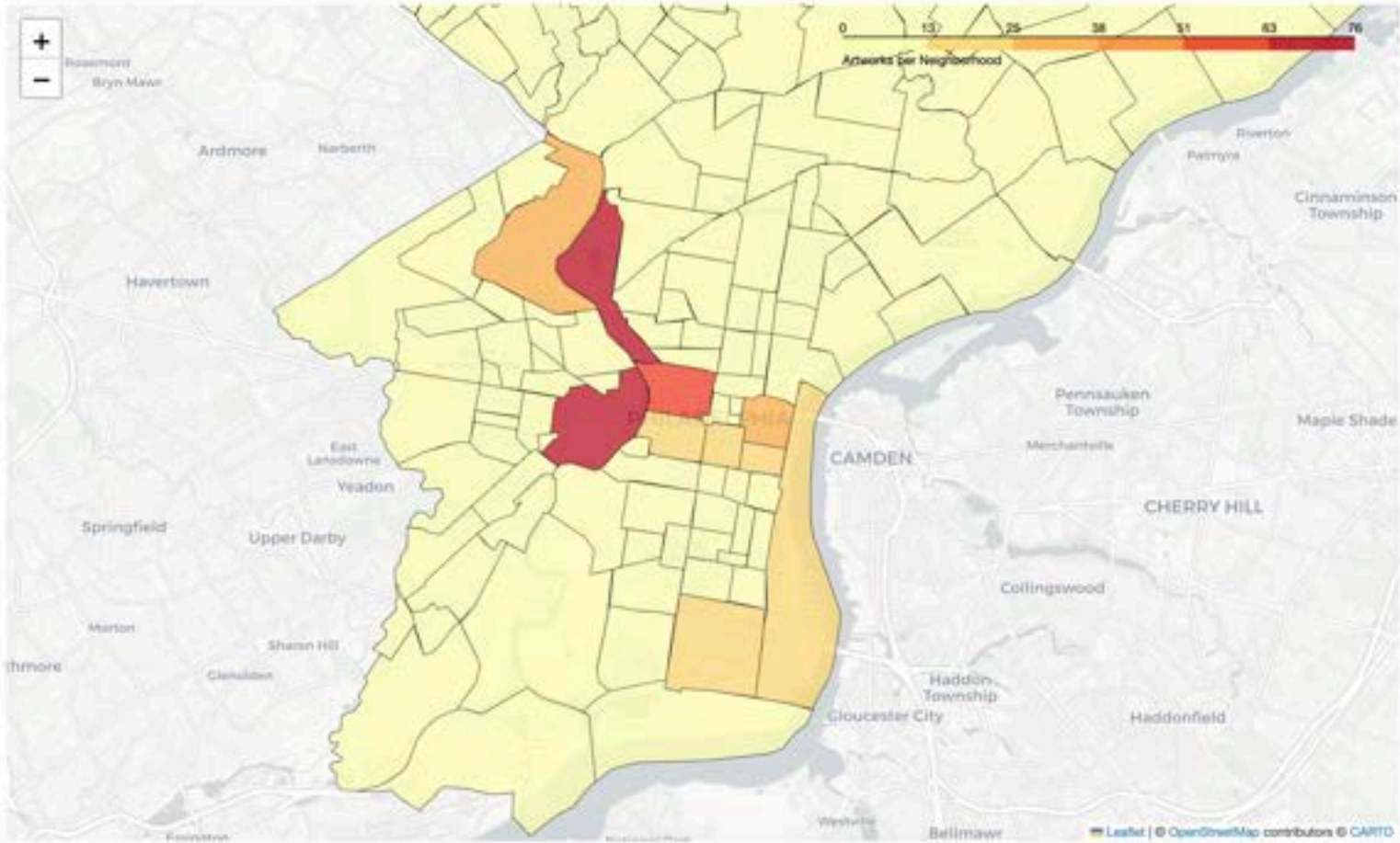
A **choropleth** colors geographic areas by a numeric value. We count artworks per neighborhood using a **spatial join**.

```
# Spatial join — count artworks per neighborhood
art_gdf = gpd.GeoDataFrame(art_df, geometry=gpd.points_from_xy(art_df["longitude"], art_df["latitude"]), crs="EPSG:4326"
art_counts = gpd.sjoin(art_gdf, neighborhoods, how="left", predicate="within").groupby("MAPNAME").size().reset_index(nar

# Choropleth layer
folium.Choropleth(
    geo_data=neighborhoods_with_art,
    data=neighborhoods_with_art,
    columns=["MAPNAME", "artwork_count"],
    key_on="feature.properties.MAPNAME",
    fill_color="YlOrRd",
    fill_opacity=0.75,
    legend_name="Artworks per Neighborhood"
).add_to(m)
```

Top neighborhoods: East Park (76), University City (64), Logan Square (52)

```
    fill_opacity=0.75,
    line_weight=0.5,
    legend_name="Artworks per Neighborhood",
    name="Artwork Density"
).add_to(m_choro)

m_choro
```

[13]:

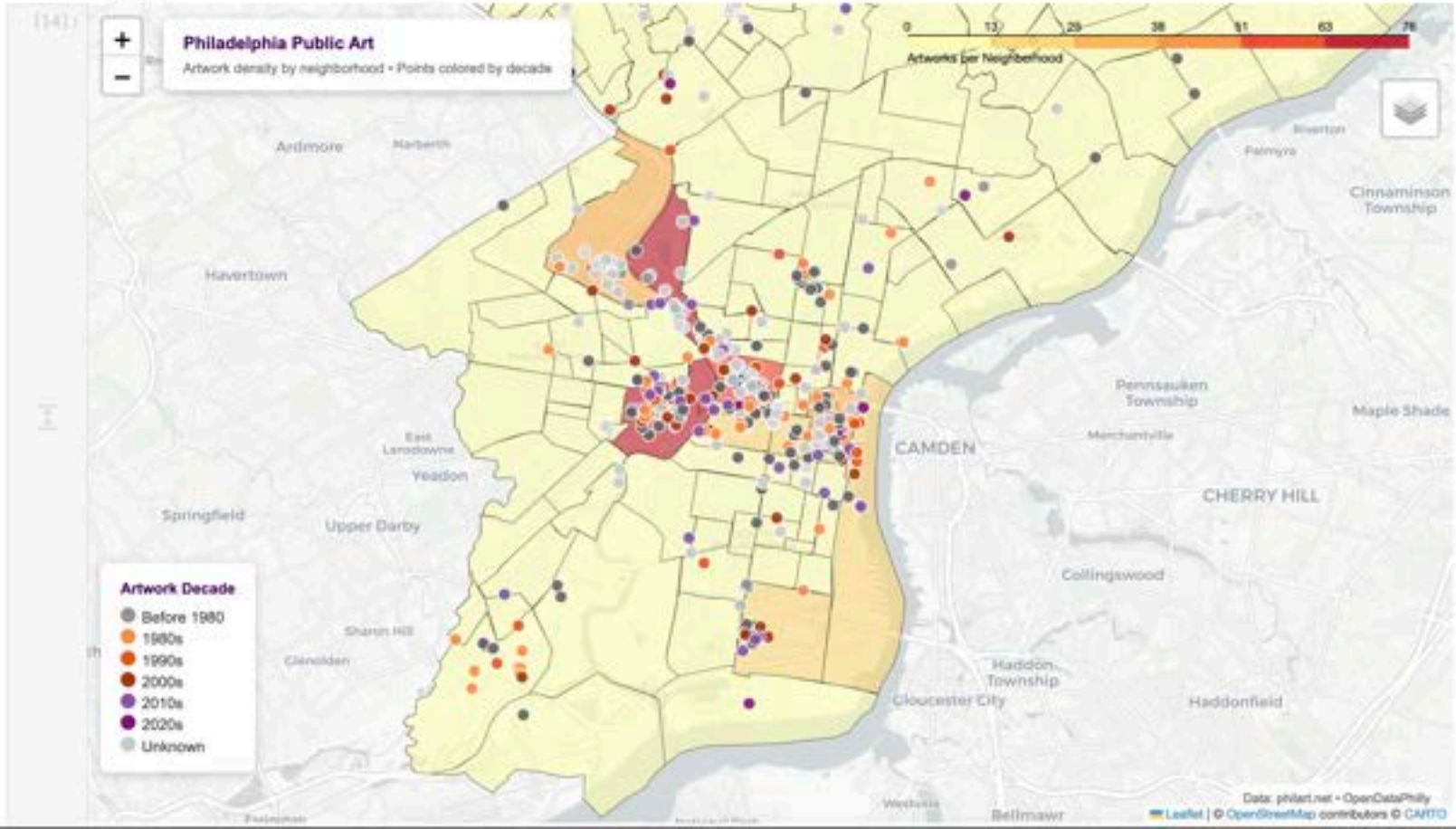


Step 8 — Final Polished Map

Combine all layers into one map with a **title**, **decade legend**, and **layer control**.

```
m_final = folium.Map(location=[39.952, -75.165], zoom_start=12, tiles="CartoDB Positron")

# 1. Choropleth base
folium.Choropleth(...).add_to(m_final)

# 2. Artwork points with popups (in a FeatureGroup for toggling)
art_layer = folium.FeatureGroup(name="Public Art (by decade)")
for _, row in art_df.iterrows():
    folium.CircleMarker(...).add_to(art_layer)
art_layer.add_to(m_final)

# 3. Custom HTML legend (bottom-left corner)
legend_html = """<div style="position:fixed; bottom:40px; left:10px; ...">...</div>"""
m_final.get_root().html.add_child(folium.Element(legend_html))

# 4. Layer toggle control
folium.LayerControl().add_to(m_final)
m_final
```

```
"""
m_final.get_root().html.add_child(folium.Element(source_html))

# Layer control (toggle layers on/off)
folium.LayerControl().add_to(m_final)

m_final
```



Step 9 — Export to HTML

Folium generates pure Leaflet.js HTML — no widgets, no server, no dependencies.

```
m_final.save("index.html")
```

Preview locally

```
python -m http.server 8000
```

Open `http://localhost:8000/index.html`

Publish on GitHub Pages

1. Create a new GitHub repository
2. Upload `index.html` to the root
3. Settings → Pages → Source → `main / root`
4. Live at `yourusername.github.io/yourrepo/`

> Pure Leaflet.js HTML — no Python server needed. Share via link, email, or embed with `<iframe>`.

Workshop Summary

Step	Skill
1–2	Create a map, set center, zoom, and basemap
3	Load a GeoJSON polygon layer from a URL
4	Load point data from a CSV file
5	Color points by a data attribute (decade)
6	Add HTML popups and hover tooltips
7	Spatial join + choropleth with `folium.Choropleth`
8	Combine layers, legend, title into a final map

Learn More

leafmap: leafmap.org · Notebooks gallery

Folium: python-visualization.github.io/folium

Data: OpenDataPhilly · philart.net

Standards: STAC specification · Cloud-Optimized GeoTIFF

GitHub · Report issues

Contact us at:
felipe.valdez@temple.edu

Visit our guides:
https://guides.temple.edu/gis-mapping